VELLORE INSTITUTE OF TECHNOLOGY, VELLORE

School of Computer Science and Engineering (SCOPE)

MINI PROJECT REPORT

<Project Title>

Course Code: BCSE302P – Database Systems Laboratory

Submitted by

S. No.	Name	Registration Number
1		
2		
3		
4		

(Maximum 4 members. Individual reports carry higher weightage. Delete unused rows.)

Under the guidance of

Dr. G. M. Karthik

Associate Professor, School of Computer Science and Engineering (SCOPE)

Vellore Institute of Technology, Vellore

Date of Submission: ____ / ____ / 2026

CERTIFICATE

This is to certify that the Mini Project Report titled "**<Project Title>**" submitted by **<Name 1 (Reg. No.)>, <Name 2 (Reg. No.)>, <Name 3 (Reg. No.)>, <Name 4 (Reg. No.)>** in partial fulfilment of the requirements of the course BCSE302P – Database Systems Laboratory, is a record of original work carried out under my supervision during the Fall Semester 2026–27.

Signature of the Faculty

Dr. G. M. Karthik

Associate Professor, SCOPE

VIT, Vellore

Place: Vellore

Date:

ABSTRACT

Write 150–200 words in a single paragraph. State the problem, the application domain, the DBMS used (e.g., Oracle / MySQL / PostgreSQL), the key database objects implemented (tables, functions, procedures, packages, triggers, cursors), whether a front-end is included, and the main outcome. Avoid citations and code in the abstract.

Keywords: *<4–6 keywords, e.g., DBMS, PL/SQL, Normalization, Triggers, ER Model>*

TABLE OF CONTENTS

CERTIFICATE 2

ABSTRACT 3

TABLE OF CONTENTS 4

1. INTRODUCTION 6

    1.1 Problem Statement 6

    1.2 Objectives 6

    1.3 Importance of DBMS 6

    1.4 Scope of the Project 6

2. SYSTEM REQUIREMENTS 7

    2.1 Hardware Requirements 7

    2.2 Software Requirements 7

3. DATABASE DESIGN 8

    3.1 Entity–Relationship (ER) Diagram 8

    3.2 Relational Schema 8

    3.3 Schema Definition (DDL) 8

    3.4 Normalization 8

4. IMPLEMENTATION 10

    4.1 Functions 10

    4.2 Procedures 10

    4.3 Packages 10

    4.4 Triggers 10

    4.5 Cursors 11

5. FRONT-END (OPTIONAL – FOR HIGHER MARKS) 12

    5.1 User Interface 12

    5.2 Integration with the DBMS 12

6. VERIFICATION & TESTING 13

6.1 Test Cases 13
6.2 Output Screenshots 13
6.3 Validation of Functionality 13
7. RESULTS & DISCUSSION 14
7.1 Achievements 14
7.2 Challenges Faced 14
7.3 Possible Improvements 14
8. CONCLUSION 15
8.1 Summary of Learning Outcomes 15
8.2 Future Scope 15
9. REFERENCES 16
10. APPENDICES 17
Appendix A – Additional Code 17
Appendix B – Additional Screenshots 17

(Right-click here in Word → Update Field → Update entire table, after completing the report.)

1. INTRODUCTION

1.1 Problem Statement

Describe the real-world problem your system addresses, who the users are, and why the existing approach (manual records, spreadsheets, etc.) is inadequate.

1.2 Objectives

- *Objective 1 – e.g., design a normalized relational schema for …*
- *Objective 2 – e.g., implement business rules using PL/SQL triggers and procedures*
- *Objective 3 – e.g., provide a web/form interface for CRUD operations*

1.3 Importance of DBMS

Explain why a DBMS is the right solution here: data integrity, reduced redundancy, concurrency control, security, recovery, and query efficiency — relate each to your application.

1.4 Scope of the Project

State what is included and what is explicitly out of scope.

2. SYSTEM REQUIREMENTS

2.1 Hardware Requirements

Component	Specification
Processor	e.g., Intel Core i5 / AMD Ryzen 5 or above
RAM	e.g., 8 GB
Storage	e.g., 256 GB SSD
Other	e.g., Android device (for mobile-based projects)

2.2 Software Requirements

Category	Tool / Version
Operating System	e.g., Windows 11 / Ubuntu 24.04
DBMS	e.g., Oracle Database 21c XE / MySQL 8.x / PostgreSQL 16
Query / IDE Tool	e.g., SQL Developer / MySQL Workbench / pgAdmin
Front-End Language	e.g., HTML, CSS, JavaScript / Python (Flask) / PHP / Java
Connectivity	e.g., JDBC / cx_Oracle / mysql-connector
Other Tools	e.g., draw.io for ER diagram, Git

3. DATABASE DESIGN

3.1 Entity–Relationship (ER) Diagram

List the entities, their key attributes, and relationships with cardinality (1:1, 1:N, M:N) and participation constraints before presenting the diagram.

[Insert ER Diagram here]

Figure 3.1: ER Diagram of <Project Title>

3.2 Relational Schema

Provide one table per relation, as in the example below. Mention PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK and DEFAULT constraints.

Table 3.1: <TABLE_NAME>

Attribute	Data Type	Constraint	Description
student_id	NUMBER(10)	PRIMARY KEY	Unique student identifier
name	VARCHAR2(50)	NOT NULL	Student name
dept_id	NUMBER(5)	FOREIGN KEY → DEPT(dept_id)	Department reference

3.3 Schema Definition (DDL)

```
CREATE TABLE dept (
  dept_id    NUMBER(5) PRIMARY KEY,
  dept_name  VARCHAR2(40) NOT NULL UNIQUE
);
-- Add CREATE TABLE statements for all relations
```

3.4 Normalization

Show the functional dependencies and the step-by-step decomposition: Unnormalized → 1NF → 2NF → 3NF (and BCNF if applicable). For each step, state the anomaly removed and show the resulting relations.

Normal Form	**Relations**	**Justification**
1NF		Atomic values; no repeating groups
2NF		No partial dependency on a composite key
3NF		No transitive dependency
BCNF		Every determinant is a candidate key

4. IMPLEMENTATION

For each database object: (a) explain its purpose in the application, (b) give the code snippet, and (c) show sample execution output. Include at least one of each type.

4.1 Functions

Explain what the function computes and where it is called from.

```
CREATE OR REPLACE FUNCTION fn_name (p_id IN NUMBER)
RETURN NUMBER IS
  v_result NUMBER;
BEGIN
  -- logic
  RETURN v_result;
END;
/
```

Output / sample execution: paste a screenshot or the result below.

4.2 Procedures

Explain the business operation the procedure performs and its IN/OUT parameters.

```
CREATE OR REPLACE PROCEDURE proc_name (p_id IN NUMBER) IS
BEGIN
  -- logic
  COMMIT;
EXCEPTION
  WHEN OTHERS THEN ROLLBACK; RAISE;
END;
/
```

Output / sample execution: paste a screenshot or the result below.

4.3 Packages

Show the package specification and body, and a usage example calling its members.

```
CREATE OR REPLACE PACKAGE pkg_name AS
  PROCEDURE proc_name (p_id IN NUMBER);
  FUNCTION  fn_name   (p_id IN NUMBER) RETURN NUMBER;
END pkg_name;
/
-- Usage: EXEC pkg_name.proc_name(101);
```

Output / sample execution: paste a screenshot or the result below.

4.4 Triggers

State the triggering event (BEFORE/AFTER INSERT/UPDATE/DELETE), level (row/statement) and the rule it enforces.

```
CREATE OR REPLACE TRIGGER trg_name
BEFORE INSERT OR UPDATE ON table_name
FOR EACH ROW
BEGIN
  -- validation / audit logic using :NEW and :OLD
END;
/
```

Output / sample execution: paste a screenshot or the result below.

4.5 Cursors

Explain why a cursor is needed (row-by-row processing) and whether it is implicit, explicit, or parameterized.

```
DECLARE
  CURSOR c_rec IS SELECT * FROM table_name;
BEGIN
  FOR r IN c_rec LOOP
    DBMS_OUTPUT.PUT_LINE(r.column_name);
  END LOOP;
END;
/
```

Output / sample execution: paste a screenshot or the result below.

5. FRONT-END (OPTIONAL – FOR HIGHER MARKS)

Required for 9 marks (webpage/form) and 10 marks (complex or mobile-based application).

5.1 User Interface

[Insert screenshot of webpage / form / mobile screen]

Figure 5.1: <Screen name>

5.2 Integration with the DBMS

Explain the architecture (front-end → connector/API → database), the connection method, and how front-end actions invoke your procedures, functions and triggers. Include a short connection code snippet.

6. VERIFICATION & TESTING

6.1 Test Cases

TC ID	Description	Input	Expected Output	Actual Output	Status
TC01	Insert valid record				Pass / Fail
TC02	Insert violating constraint		Error raised		
TC03	Trigger fires on update				
TC04					

6.2 Output Screenshots

[Insert output screenshot]

Figure 6.1: <Description of output>

6.3 Validation of Functionality

Summarise how the test results confirm that constraints, triggers, procedures and the front-end behave correctly, including negative/edge cases.

7. RESULTS & DISCUSSION

7.1 Achievements

What the system successfully does; relate back to the objectives in Section 1.2.

7.2 Challenges Faced

Technical difficulties (e.g., mutating-table errors, connectivity issues) and how they were resolved.

7.3 Possible Improvements

Enhancements in performance (indexing), security (roles, privileges), usability, etc.

8. CONCLUSION

8.1 Summary of Learning Outcomes

Key DBMS concepts learned and applied through this project.

8.2 Future Scope

How the system could be extended — e.g., cloud deployment, analytics, mobile app, multi-user access control.

9. REFERENCES

Use a consistent citation style (IEEE recommended). Examples:

[1] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York, NY, USA: McGraw-Hill, 2019.

[2] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Boston, MA, USA: Pearson, 2016.

[3] Oracle Corporation, “PL/SQL Language Reference,” Oracle Help Center. [Online]. Available: https://docs.oracle.com/ (accessed <date>).

10. APPENDICES

Appendix A – Additional Code

Complete source code not included in the main body.

Appendix B – Additional Screenshots

Screenshots not included in the main body, each with a figure caption.